10 - Colecciones y genéricos

Pregunta número 1

In the Java collections framework a Set Is:

Seleccione una:

- a. An ordered collection that can contain duplicate elements.
- b. A collection that cannot contain duplicate elements.
- c. An object that maps value key sets and cannot contain values Duplicates.

Respuesta: Para resolver este ejercicio, hay que entender primero que un Set modela el concepto matemático de conjunto, y en un conjunto cada elemento aparece una sola vez. Esa es justamente la característica que define a la interfaz Set dentro del framework de colecciones: no admite elementos duplicados. Si intentamos agregar un elemento que ya existe (Java lo determina usando equals() y hashCode()), el método add() simplemente regresa false y la colección se queda igual, sin lanzar ningún error. El inciso a lo descarto porque está describiendo a una List, que es la colección ordenada por índices que sí permite repetir elementos. El inciso c tampoco aplica, ya que habla de un objeto que asocia claves con valores, lo cual es la definición de un Map; además, en un Map lo que no se puede repetir son las claves, no los valores. Por lo tanto, la única descripción que le corresponde a un Set es la de una colección que no puede contener elementos duplicados.

Pregunta número 2

¿Cuál de las siguientes afirmaciones sobre ArrayList es verdadera?

Seleccione una:

- a. ArrayList permite elementos duplicados.
- b. ArrayList no mantiene el orden de inserción de los elementos.
- c. Los elementos de ArrayList se almacenan en una estructura de árbol.
- d. ArrayList puede almacenar tipos primitivos directamente.

Respuesta: En este ejercicio considero importante recordar cómo está construido un ArrayList por dentro: es una implementación de la interfaz List respaldada por un arreglo dinámico que va creciendo conforme lo necesitamos. Como es una List, permite elementos duplicados sin ningún problema; podemos agregar la misma cadena o el mismo número todas las veces que queramos y cada uno ocupará su propio índice. Las demás afirmaciones no se sostienen. Un ArrayList sí mantiene el orden de inserción, porque cada elemento nuevo se coloca al final y conserva su posición, así que el inciso b es falso. Tampoco usa una estructura de árbol; eso es

propio de clases como TreeSet o TreeMap, que internamente usan un árbol rojo-negro, mientras que ArrayList trabaja sobre un arreglo, por lo que el inciso c queda fuera. Finalmente, el inciso d funciona como una trampa, ya que un ArrayList solo puede almacenar objetos. No podemos declarar un ArrayList<int>; lo que hacemos es usar la clase envoltorio Integer, y cuando escribimos lista.add(5), el autoboxing convierte ese int en un Integer antes de guardarlo.

Pregunta número 3

¿Cuál es la característica que comparten Array y Arraylist?

Seleccione una:

- a. Pueden almacenar valores nulos.
- b. Ambos procesos ocurren en tiempo constante.
- c. Es posible que tengan valores duplicados.
- d. Array y ArrayList no garantizan la presencia de elementos ordenados.

Respuesta: Este ejercicio tiene un detalle interesante, porque varias opciones parecen razonables a primera vista, así que vale la pena buscar la característica que sea verdadera siempre, para ambos y sin excepciones. Tanto un arreglo como un ArrayList organizan sus elementos por índice, y nada impide que dos posiciones distintas guarden el mismo valor; podemos tener un int[] con {3, 3, 3} o un ArrayList con "Ana" repetido varias veces. Por eso, la posibilidad de tener valores duplicados es una característica que comparten sin importar el tipo de dato que almacenen. El inciso a suena tentador, pero yo lo descarto porque no siempre se cumple: un ArrayList sí acepta null, pero un arreglo de tipo primitivo, como un int[] o un double[], jamás puede guardar un null; solo los arreglos de tipos de referencia lo permiten. El inciso b es demasiado general y además es falso, ya que no todas las operaciones son de tiempo constante (por ejemplo, insertar o eliminar en medio de un ArrayList implica recorrer los elementos). El inciso d también es incorrecto, porque ambos sí conservan el orden de sus elementos según su índice.

Pregunta número 4

¿Cuál de las siguientes no es parte del framework de Java Collections?

Seleccione una:

- a. ArrayList
- b. Map
- c. Dictionary
- d. List

Respuesta: En este ejercicio, lo importante es recordar que Dictionary es una clase abstracta que existe desde el JDK 1.0, es decir, es anterior al framework de colecciones, que llegó hasta Java 1.2. Hoy se considera una clase legacy (obsoleta), y la propia documentación de Java indica que en su lugar debemos usar la interfaz Map. Aunque la clase Hashtable hereda de Dictionary y posteriormente fue adaptada para implementar Map, Dictionary por sí misma nunca formó parte del framework. En cambio, ArrayList y List son piezas centrales de la jerarquía de colecciones. La opción que podría generar dudas es Map, porque no hereda de la interfaz Collection; sin embargo, sí forma parte del Java Collections Framework, ya que es una de sus interfaces principales junto con sus implementaciones (HashMap, TreeMap, LinkedHashMap, etc.). Por lo tanto, el único de la lista que no pertenece al framework es Dictionary.

Pregunta número 5

¿Qué método en java.lang.Object debe ser sobrescrito si se sobrescribe el método hashCode()?

Seleccione una:

- a. clone()
- b. compareTo()
- c. copy()
- d. equals()
- e. sort()

Respuesta: En este ejercicio hay que recordar que los métodos equals() y hashCode() siempre van de la mano, y yo diría que esta es una de las reglas más importantes al hablar de colecciones. El contrato de Object establece que, si dos objetos son iguales según equals(), obligatoriamente deben devolver el mismo valor en hashCode(). Si sobrescribimos solo uno de los dos, rompemos ese contrato y las colecciones basadas en hash, como HashMap o HashSet, empiezan a comportarse

de forma inesperada: pueden guardar duplicados o no encontrar elementos que sí están. Por eso, si sobrescribimos hashCode(), también debemos sobrescribir equals(), y viceversa. Revisando las demás opciones, clone() sí existe en Object, pero no tiene ninguna relación con el hash. compareTo() ni siquiera pertenece a Object, sino a la interfaz Comparable, y sirve para ordenar, no para determinar igualdad en un hash. Finalmente, copy() y sort() no son métodos de java.lang.Object, así que quedan descartados de inmediato.

Pregunta número 6

What is the output?

```
import java.util.ArrayList;
import java.util.List;
public class Test {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        printList(list);
        System.out.println("Size: " + list.size());
        System.out.println("Contains 'Banana': " + list.contains("Banana"));
        System.out.println("Element at index 1: " + list.get(1));
    }
    private static void printList(List<String> list) {
        for (String element : list) {
            System.out.println(element);
        }
    }
}
```

Seleccione una:

- a. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana
- b. Apple Banana Size: 2 Contains 'Banana': false Element at index 1: Banana
- c. Apple Banana Size: 2 Contains 'Banana': true Element at index 1: Banana
- d. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Cherry

Respuesta: En este ejercicio hay que seguir el código paso a paso para entender lo que esta sucediendo. Primero creamos una lista y le agregamos "Apple", "Banana" y "Cherry", en ese orden. Después llamamos a printList(list), que recorre la lista con un for-each e imprime cada elemento en su propia línea, por lo que vemos Apple, Banana y Cherry. Luego, list.size() regresa 3, porque agregamos tres elementos y no eliminamos ninguno. De ahí pasamos a list.contains("Banana"), que regresa true,

ya que contains() usa equals() para comparar y "Banana" sí está en la lista. Finalmente, list.get(1) regresa el elemento del índice 1; como los índices empiezan en 0, Apple es el índice 0 y Banana es el índice 1. Juntando todo, la salida es Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana (en el inciso aparece en una sola línea, pero en consola cada println genera su propio renglón). Los demás incisos intentan confundirnos: los que omiten Cherry y dicen Size: 2 inventan que se perdió un elemento, y el que dice Cherry en el índice 1 es para quien olvide que los índices empiezan en 0.

Pregunta número 7

Dado el siguiente código, ¿cuál es el resultado?

```
import java.util.*;

public class Ordenamiento {
    public static void main(String[] args) {
        List<Integer> numeros = new ArrayList<Integer>();
        numeros.add(8);
        numeros.add(5);
        numeros.add(3);
        numeros.add(6);

        Collections.sort(numeros);
        System.out.println(numeros);
    }
}
```

Seleccione una:

- a. [8, 5, 3, 6]
- b. [3, 5, 6, 8]
- c. Compilation error
- d. [8, 6, 5, 3]

Respuesta: Hay que analizar este ejercicio con cuidado para entender si el código compila, y si no hay ningún problema. Primero se importa todo java.util, se declara una List<Integer> y se agregan los valores 8, 5, 3 y 6. La parte importante es Collections.sort(numeros). Cuando llamamos a este método sin pasarle un Comparator, ordena la lista según el orden natural de sus elementos. Como Integer implementa Comparable, su orden natural es el numérico ascendente, de menor a mayor. Por lo tanto, la lista queda como [3, 5, 6, 8], y al imprimirla con println se usa el toString() de la lista, que muestra los elementos entre corchetes y separados por comas. El inciso a sería el resultado si el sort no hiciera nada (es el orden de inserción); el inciso d sería el orden descendente, que solo obtendríamos con algo

como Collections.reverseOrder(), y el inciso c no aplica porque el código es completamente válido.

Pregunta número 8

¿Cuál de las siguientes implementaciones de código utiliza correctamente una clase genérica en Java?

Seleccione una o más de una:

a.
```
public class Box {
    private Object value;
    public Box(Object value) {
        this.value = value;
    }
    public Object getValue() {
        return value;
    }
}
```

b.
```
public class Box {
    private int value;
    public Box(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
```

c.
```
public class Box<T> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

d.
```
public class Box<T extends Number> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

Respuesta: En este ejercicio hay que recordar que una clase genérica es aquella que declara uno o más parámetros de tipo entre los símbolos < > justo después de su nombre, para que el tipo concreto se decida al momento de usarla. Con esa definición en mente, revisemos cada fragmento. El inciso a funciona y compila, pero no es genérico: usa Object para poder guardar cualquier cosa. El problema es que perdemos la seguridad de tipos, ya que al llamar a getValue() tenemos que hacer un cast manual y, si nos equivocamos, el error aparece hasta tiempo de ejecución. El inciso b tampoco es genérico, ya que está atado de forma fija al tipo int. El inciso c es el ejemplo clásico de una clase genérica: declara el parámetro de tipo T en Box<T> y lo usa en el atributo, en el constructor y en el tipo de retorno; así, un Box<String> solo aceptará y devolverá String, y el compilador lo verifica por nosotros. El inciso d también es correcto, porque sigue siendo una clase genérica, solo que con un tipo acotado (bounded type): T extends Number restringe a que T sea Number o alguna de sus subclases, como Integer o Double. Por eso, las dos implementaciones que usan correctamente genéricos son la c y la d.

Pregunta número 9

¿Qué clase puede proveer las siguientes capacidades?: no permitir valores duplicados, mantener el orden en el cual los elementos fueron insertados, proveer iteración rápida y acceso aleatorio eficiente.

Seleccione una:

- a. TreeSet
- b. LinkedHashSet
- c. HashSet
- d. ArrayList
- e. LinkedHashMap

Respuesta: En este ejercicio hay que ir filtrando las opciones con cada requisito de la pregunta para poder elegir el inciso correcto. El primero es no permitir duplicados, lo que nos deja solo con las implementaciones de Set (TreeSet, LinkedHashSet y HashSet); ArrayList queda fuera porque acepta duplicados, y LinkedHashMap también, porque es un Map que almacena pares clave-valor, no elementos individuales. El segundo requisito es mantener el orden de inserción, y aquí se cae HashSet, que no garantiza ningún orden, y también TreeSet, que ordena los elementos según su orden natural o un Comparator, pero no según el momento en que se insertaron. La única que queda es LinkedHashSet, que combina una tabla hash con una lista doblemente enlazada que recorre todas sus entradas. Esa lista enlazada es la que conserva el orden de inserción y hace que la iteración sea rápida, ya que el recorrido es proporcional al número de elementos y no a la capacidad de la tabla. Por su parte, la tabla hash le da el acceso eficiente: localizar

un elemento con contains() o eliminarlo toma tiempo constante en promedio. Un detalle que debemos tener claro es que un Set no tiene acceso por índice como una List, así que el acceso aleatorio eficiente de la pregunta se refiere a poder localizar cualquier elemento rápidamente gracias al hash.

Pregunta número 10

¿Cuál será el resultado de intentar compilar y ejecutar el siguiente fragmento de código?

```
import java.util.*;

public class ComparableTest {
    public static void main(String[] args) {
        List<Item> items = new ArrayList<>();
        items.add(new Item("Apple", 3));
        items.add(new Item("Banana", 2));
        items.add(new Item("Cherry", 1));
        Collections.sort(items);
        for (Item item : items) {
            System.out.println(item.getName() + ": " + item.getPrice());
        }
    }
}

class Item implements Comparable<Item> {
    private String name;
    private int price;

    public Item(String name, int price) {
        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public int getPrice() {
        return price;
    }

    @Override
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

Seleccione una:

- a. Error de compilación debido a una implementación incorrecta de Comparable.
- b. Imprimirá los artículos ordenados por precio.
- c. Error de tiempo de ejecución debido a una implementación incorrecta de Comparable.
- d. Imprimirá los artículos ordenados por nombre.

Respuesta: En este ejercicio, lo primero que debemos verificar es si hay algún problema de compilación con la implementación de Comparable, porque los incisos a y c apuntan justamente a eso. La clase Item implementa Comparable<Item> y sobrescribe compareTo(Item other) con la firma exacta que exige la interfaz, regresando un int; la anotación @Override es válida y todo compila sin errores. Tampoco hay nada que provoque una excepción en tiempo de ejecución, ya que ningún nombre es null. Ahora bien, lo que define el orden es lo que regresa compareTo(): en este caso, this.name.compareTo(other.name), es decir, se comparan los nombres usando el orden alfabético de los String. El precio nunca se toma en cuenta. Cuando Collections.sort(items) ordena la lista, usa ese criterio y deja los artículos como Apple, Banana y Cherry. Viendo con cuidado, podemos notar que hay una pequeña trampa visual: con estos datos, el orden alfabético coincide con un orden descendente por precio (3, 2, 1), y eso podría hacernos pensar que se ordenó por precio. Pero si fuera por precio ascendente, el primero sería Cherry con 1. Por eso, lo correcto es afirmar que imprimirá los artículos ordenados por nombre.

Pregunta número 11

What is the output?

```
import java.util.*;
public class ComparableTest {
    public static void main(String[] args) {
        List<Item> items = new ArrayList<>();
        items.add(new Item("Apple", 3));
        items.add(new Item("Banana", 2));
        items.add(new Item("Cherry", 1));
        Collections.sort(items);
        for (Item item : items) {
            System.out.println(item.getName() + ": " + item.getPrice());
        }
    }
}
class Item implements Comparable<Item> {
    private String name;
    private int price;
    public Item(String name, int price) {
        this.name = name;
        this.price = price;
    }
    public String getName() {
        return name;
    }
    public int getPrice() {
        return price;
    }
    @Override
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

Seleccione una:

- a. Apple: 3 Banana: 2 Cherry: 1
- b. Compilation error
- c. Apple: 3 Cherry: 1 Banana: 2
- d. Cherry: 1 Banana: 2 Apple: 3

Respuesta: Este ejercicio es exactamente el mismo de la pregunta anterior, pero ahora nos piden la salida concreta en lugar de tener que describirla. Ya sabemos que compila, porque compareTo() está bien implementado, así que el inciso b se descarta. El criterio de ordenamiento que definimos en compareTo() es el nombre del artículo en orden alfabético, así que después de Collections.sort(items) la lista queda en el orden Apple, Banana, Cherry. El for-each recorre la lista e imprime en cada línea el nombre, dos puntos y el precio, por lo que obtenemos Apple: 3, Banana: 2 y Cherry: 1. El inciso d sería el resultado si el compareTo() comparara precios en orden ascendente, y el inciso c no corresponde a ningún criterio lógico. Siento que lo más útil e importante de este tipo de preguntas es acostumbrarnos a ir directamente al método compareTo() o compare() para identificar el criterio real de ordenamiento, en lugar de dejarnos llevar por cómo se ven los datos.

Pregunta número 12

What is the result?

```
import java.util.Comparator;
import java.util.Arrays;

class MySort implements Comparator<Integer> {
    public int compare(Integer x, Integer y) {
        return y.compareTo(x);
    }
}

public class Main {
    public static void main(String[] args) {
        Integer[] primes = {2, 7, 5, 3};
        MySort ms = new MySort();
        Arrays.sort(primes, ms);
        for (Integer p2 : primes) {
            System.out.print(p2 + "");
        }
    }
}
```

Seleccione una:

- a. Compilation fails
- b. 2 7 5 3
- c. 2 3 5 7
- d. 7 5 3 2

Respuesta: La clave de este ejercicio está en una sola línea: return y.compareTo(x). El contrato de compare(x, y) dice que debe regresar un número negativo si x va antes que y, cero si son iguales y positivo si x va después. Lo natural, para un orden ascendente, sería escribir x.compareTo(y); pero aquí los argumentos están invertidos, así que el orden también se invierte y obtenemos un orden descendente. Revisando la compilación, todo es correcto: MySort implementa Comparator<Integer> con la firma adecuada, y Arrays.sort(primes, ms) es válido porque primes es un Integer[] (un arreglo de objetos). Este es un detalle importante, ya que si el arreglo fuera de tipo int[], no podríamos pasarle un Comparator y el código no compilaría. Al ordenar {2, 7, 5, 3} de mayor a menor, obtenemos 7, 5, 3 y 2. Un pequeño detalle que me llamó la atención es que se usa print(p2 + ""), que no agrega ningún espacio, así que en la consola se observa 7532 todo junto; el inciso lo muestra con espacios, pero corresponde a la misma secuencia.

Pregunta número 13

What is the result?

```
import java.util.Arrays;
import java.util.Comparator;
class SortLongs implements Comparator<Long> {
    @Override
    public int compare(Long a, Long b) {
        return b.compareTo(a);
    }
}
public class Test {
    public static void main(String[] args) {
        Long[] values = {20L, 70L, 50L, 30L};
        SortLongs sort = new SortLongs();
        Arrays.sort(values, sort);
        System.out.println(Arrays.toString(values));
    }
}
```

Seleccione una:

- a. An ArrayIndexOutOfBoundsException is thrown.
- b. [20, 70, 50, 30]
- c. [20, 30, 50, 70]
- d. [70, 50, 30, 20]
- e. Compilation fails.

Respuesta: Este código nos va a resultar muy familiar, porque sigue el mismo patrón, pero con Long. En el método compare(Long a, Long b) se regresa b.compareTo(a), es decir, se invierte el orden natural y los valores se ordenan de mayor a menor. Me parece prudente revisar si todo compila correctamente: SortLongs implementa Comparator<Long> correctamente; el @Override es válido y los literales 20L, 70L, 50L y 30L se convierten automáticamente a objetos Long por el autoboxing, así que el arreglo Long[] se crea sin problema. Arrays.sort(values, sort) ordena el arreglo con nuestro comparador y nos queda 70, 50, 30, 20. Finalmente, Arrays.toString(values) genera la representación con corchetes y comas, imprimiendo [70, 50, 30, 20]. No hay ningún acceso fuera de los límites, por lo que la ArrayIndexOutOfBoundsException no tiene sentido; el inciso b es simplemente el orden original y el inciso c sería el orden ascendente que obtendríamos con a.compareTo(b).

Pregunta número 14

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        Integer[] valores = {4, 8, 1, 6};
        Arrays.sort(valores, Collections.reverseOrder());

        for (Integer v : valores) {
            System.out.println(v);
        }
    }
}
```

Seleccione una:

- a. 4,8,1,6
- b. RuntimeException
- c. 8,6,4,1
- d. 1,4,6,8

Respuesta: Lo primero que debemos tener en cuenta en este ejercicio es ver que Collections.reverseOrder() nos regresa un Comparator que impone el orden inverso al orden natural de los elementos. Como el arreglo es de tipo Integer[], es decir, de objetos, podemos pasárselo a Arrays.sort() junto con ese comparador sin ningún problema, y el resultado es un ordenamiento de mayor a menor. Partimos de {4, 8, 1, 6}; ordenado de forma descendente nos queda 8, 6, 4, 1, y el for-each lo imprime con un número por línea. No hay motivo para una RuntimeException, ya que ningún

elemento es null. El inciso a es el arreglo original sin ordenar y el inciso d es el orden ascendente que obtendríamos con un simple Arrays.sort(valores). Como dato importante, este código no funcionaría con un int[], porque los comparadores solo trabajan con objetos.

Pregunta número 15

Dado el siguiente código. ¿Cuál es el resultado?

```
public class Ordenamiento {
    public static void main(String[] args) {
        List<String> names = new ArrayList<String>();
        names.add("Mariel");
        names.add("Alejandro");
        names.add("Elo");
        names.add("Nadia");

        Comparator<String> listNames = Collections.reverseOrder();
        Collections.sort(names, listNames);

        System.out.println(names);
    }
}
```

Seleccione una:

- a. Nadia, Mariel, Elo, Alejandro
- b. Compilation error
- c. Mariel, Alejandro, Elo, Nadia
- d. Alejandro, Elo, Mariel, Nadia

Respuesta: En este ejercicio volvemos a usar Collections.reverseOrder(), pero ahora aplicado a una lista de String. Lo primero que podríamos preguntarnos es si la línea Comparator<String> listNames = Collections.reverseOrder() compila, y sí lo hace: reverseOrder() es un método genérico y el compilador infiere que el tipo es String gracias a la variable que recibe el resultado, así que el inciso b se descarta. El orden natural de los String es alfabético, por lo que su orden inverso va de la Z a la A. Si tomamos la primera letra de cada nombre, tenemos N de Nadia, M de Mariel, E de Elo y A de Alejandro; de mayor a menor, la lista queda como Nadia, Mariel, Elo, Alejandro. Al imprimirla con println veremos [Nadia, Mariel, Elo, Alejandro], con los corchetes que agrega el toString() de la lista, aunque el inciso los omite. El inciso c es el orden de inserción, que solo veríamos si no se hubiera ordenado nada, y el inciso d es el orden alfabético normal, que obtendríamos con Collections.sort(names) sin el comparador.

Pregunta número 16

¿Cuáles de los siguientes fragmentos de código implementan correctamente Comparator para ordenar una lista de objetos Person por su edad en orden ascendente? (Elige dos)

```
import java.util.Comparator;

class Person {
    private String name;
    private int age;
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public int getAge() {
        return age;
    }
    public String getName() {
        return name;
    }
}
```

Seleccione una o más de una:

a.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
}
```

b.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getName().compareTo(p2.getName());
    }
}
```

c.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

d.
```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p2.getAge() - p1.getAge();
    }
}
```

Respuesta: En este ejercicio, lo primero que tenemos que recordar es que, para que un Comparator ordene de forma ascendente por edad, el método compare(p1, p2) debe regresar un número negativo cuando p1 tenga menos edad que p2, cero cuando tengan la misma edad y un número positivo cuando p1 sea mayor. El inciso a cumple exactamente con esto: si p1 tiene 20 años y p2 tiene 30, la resta p1.getAge() - p2.getAge() da -10, un número negativo, así que p1 se coloca primero. Es una técnica muy usada, aunque yo siempre tengo presente que, con valores enteros muy extremos, la resta podría desbordarse; con edades eso no ocurre. El inciso c es la forma más recomendada: Integer.compare(p1.getAge(), p2.getAge()) hace la misma comparación, pero sin ningún riesgo de desbordamiento, y regresa -1, 0 o 1 según corresponda. El inciso b compila perfectamente, pero ordena por nombre en orden alfabético, no por edad, así que no cumple con lo que pide la pregunta. El inciso d sí ordena por edad, pero al invertir la resta (p2 - p1), el resultado es un orden descendente, de mayor a menor edad. Por eso, las dos implementaciones correctas son la a y la c.

Pregunta número 17

What is the result?

```
import java.util.*;
public class App{
    public static void main(String[]args){
        List p = new ArrayList();
        p.add(7);
        p.add(1);
        p.add(5);
        p.add(1);
        p.remove(1);
        System.out.println(p);
    }
}
```

Seleccione una:

- a. [7, 1]
- b. [7, 1, 5, 1]
- c. [7, 5, 1]
- d. [7, 5]

Respuesta: En este ejercicio, el primer detalle que noto es que la lista se declara sin genéricos (raw type). Esto compila, aunque el compilador nos muestre advertencias, y los números que agregamos se convierten automáticamente en

objetos Integer. Después de los cuatro add(), la lista queda como [7, 1, 5, 1]. Ahora viene la parte interesante: p.remove(1). La interfaz List tiene dos versiones de este método: remove(int index), que elimina por posición, y remove(Object o), que elimina por valor. Como estamos pasando un literal int, el compilador elige la versión que coincide exactamente con el tipo primitivo, que es remove(int index), sin necesidad de hacer autoboxing. Por lo tanto, se elimina el elemento que está en el índice 1, que es el primer 1, y la lista queda como [7, 5, 1]. Algo curioso es que, aun si se hubiera eliminado por valor, remove(Object) solo quita la primera coincidencia, así que el resultado habría sido el mismo; por eso el inciso d, que elimina todos los 1, es incorrecto en cualquier caso. El inciso b ignora por completo la eliminación y el inciso a no corresponde a ninguna interpretación válida. Si quisiéramos forzar la eliminación por valor, tendríamos que escribir p.remove(Integer.valueOf(1)).

Pregunta número 18

What is the result?

```
public class MyStuff {
    String name;
    MyStuff (String n) { name = n; }
    public static void main (String[] args) {
        MyStuff m1 = new MyStuff ("guitar");
        MyStuff m2 = new MyStuff ("tv");
        System.out.println (m2.equals(m1));
    }
    public boolean equals (Object o) {
        MyStuff m = (MyStuff) o;
        if (m.name != null) { return true; }
        return false;
    }
}
```

Seleccione una:

- a. The output is true and MyStuff fulfills the Object.equals() contract
- b. The output is false and MyStuff fulfills the Object.equals() contract
- c. The output is true and MyStuff does NOT fulfill the Object.equals() contract.
- d. The output is false and MyStuff does NOT fulfill the Object.equals() contract

Respuesta: En este ejercicio tenemos que contestar dos cosas: qué se imprime y si la clase cumple con el contrato de equals(). La salida es sencilla de calcular: m2.equals(m1) hace un cast de m1 a MyStuff y revisa si su name es distinto de null; como m1 tiene el nombre "guitar", la condición es verdadera y se imprime true. Eso nos deja solo con los incisos a y c. Ahora bien, el contrato de equals() exige que el

método sea reflexivo, simétrico, transitivo y consistente, y que cualquier objeto comparado con null regrese false. Esta implementación lo rompe de varias formas. Para empezar, considera iguales a dos objetos completamente distintos ("tv" y "guitar"), porque nunca compara el contenido real de ambos. Además, si el name es null, x.equals(x) regresa false, lo que rompe la reflexividad. Por si fuera poco, al comparar con null se lanza una NullPointerException en lugar de regresar false, y al recibir un objeto de otra clase se lanzaría una ClassCastException. Finalmente, la clase sobrescribe equals() sin sobrescribir hashCode(), lo que también rompe la relación entre ambos métodos. Por todo esto, la salida es true y MyStuff no cumple con el contrato de Object.equals().

Pregunta número 19

What is the result?

```
class MyKeys {
    Integer key;
    MyKeys(Integer k) { key = k; }
    public boolean equals(Object o) {
        return ((MyKeys) o).key == this.key;
    }
}

// And this code snippet:

Map m = new HashMap();
MyKeys m1 = new MyKeys(1);
MyKeys m2 = new MyKeys(2);
MyKeys m3 = new MyKeys(1);
MyKeys m4 = new MyKeys(new Integer(2));
m.put(m1, "car");
m.put(m2, "boat");
m.put(m3, "plane");
m.put(m4, "bus");
System.out.print(m.size());
```

Seleccione una:

- a. 2
- b. 3
- c. 4
- d. Compilation fails.

Respuesta: Considero que esta es una de las preguntas más importantes del tema, porque muestra lo que pasa cuando sobrescribimos equals() pero nos olvidamos de

hashCode(). Primero confirmemos que el código compila: el uso de tipos crudos (raw types) en el Map y el constructor new Integer(2), que está deprecado, solo generan advertencias, no errores, así que el inciso d queda descartado. Cuando hacemos un put() en un HashMap, lo primero que hace es calcular el hashCode() de la clave para decidir en qué bucket va a guardarla, y solo si encuentra otra clave en ese mismo bucket utiliza equals() para ver si son iguales. Como MyKeys no sobrescribe hashCode(), se usa la versión de Object, que genera un valor basado en la identidad de cada objeto. Así, m1 y m3, que lógicamente tienen la misma clave 1, terminan con hash codes distintos; el HashMap los trata como claves diferentes y ni siquiera llega a llamar a equals(). Lo mismo pasa con m2 y m4, por lo que se guardan las cuatro entradas y se imprime 4. Hay un detalle adicional que vale la pena notar: el equals() compara los Integer con ==, que compara referencias. Para los valores 1 y 2, el autoboxing reutiliza objetos de la caché de Integer, pero new Integer(2) crea un objeto nuevo, así que, aunque sobrescribiéramos hashCode(), m2 y m4 seguirían sin ser iguales y el resultado sería 3. En la forma en que está escrito el código, la respuesta es 4.